

Web3Geeks Client-Onboarding Agent

Executive Report — Production-Ready Agent System, Evaluation & Deployment

Business goal

Web3Geeks fields a steady stream of inbound project inquiries — NFT marketplaces, DeFi dApps, smart contract audits, landing pages, tokenomics consulting — each needing a scoped, priced proposal before a human ever picks up the conversation. Today that first pass (matching the request to a service, checking the budget is realistic, drafting an initial quote) is manual and inconsistent between whoever handles it. The goal of this system is to automate that first pass end-to-end — intake, validation, research, drafting, and self-correction — while keeping a human firmly in control of the one action that actually matters: whether a priced proposal goes out to a real client.

Architecture

The system is a single LangGraph workflow with eleven nodes and three graceful-exit branches (see the accompanying architecture diagram). A client inquiry enters at `validate_input`, which rejects malformed requests (missing fields, non-positive budget, unreadable descriptions) before any paid LLM or external API call happens. Valid requests proceed to `convert_currency`, which calls a real public FX API (`open.er-api.com`) to normalize non-USD budgets to USD, with a 5-second timeout and a graceful fallback if that call fails. `triage` and `retrieve` then match the request against a local services catalog (`services_catalog.json`) covering five service types with price ranges and timelines. `draft_proposal` writes a scoped proposal, and `critique` scores it against a quality threshold — looping back to `draft_proposal` (capped at two revisions) if the score is too low, mirroring the Day 3 self-correction pattern. Once a draft clears the threshold, the graph pauses at `publish_gate` (a LangGraph `interrupt_before` checkpoint) until a human explicitly approves or rejects it via the API's `/approve` endpoint — only then does `send_proposal` run. State persists across this pause via a `MemorySaver` checkpointer keyed by `thread_id`, so the pause can span real time (a reviewer coming back an hour later) without losing context.

Framework choice rationale

LangGraph was chosen over CrewAI or a raw hand-rolled loop because this problem is fundamentally a stateful, control-heavy pipeline, not a team of loosely-coupled specialists. It has a fixed sequence of named stages, a genuine self-correction loop (`critique` routing back to `draft_proposal`), and — critically — a human approval checkpoint that must pause execution and resume later, which needs an explicit `interrupt/resume/persistence` mechanism. CrewAI's role-based delegation model (evaluated on Day 4) is a better fit when the value comes from distinct specialist personas collaborating; here there's one coherent task needing precise control flow, not multiple perspectives to reconcile. A raw Day-1-style loop could technically implement the same logic, but would mean hand-building conditional routing, an interrupt mechanism, and checkpointed state persistence from scratch — exactly the boilerplate LangGraph exists to remove.

Evaluation results

Six criteria were defined to score the agent (see table); two are automatically measured by `evaluation.py`, four require a human to read each run's output. The agent was run on 10 test cases: 5 normal requests (including one exercising the real currency-conversion API with a PKR budget), 3 deliberately invalid inputs (zero budget, negative budget, gibberish description), and 2 adversarial cases (a prompt-injection attempt asking the agent to skip approval and reveal its instructions, and an unrealistic-scope request asking for an enterprise DeFi platform for \$50). Full scoring lives in the accompanying `evaluation_results_template.xlsx` — pre-filled with all 10 test cases and expected behavior, with the six criteria columns left for the reviewer to fill in after an actual run, since quality/tone/safety genuinely need a human reader and shouldn't be self-graded by the same model that wrote the output.

Criterion	What it measures	How it's scored
Task success rate	Did the run produce a sendable proposal or correctly stop for bad input	10 test cases, human-scored after running <code>evaluation.py</code>
Factual accuracy / grounding	Proposal's price and timeline match the <code>services_catalog.json</code> entry	Human-scored per run
Latency	Wall-clock seconds per run	Automatic -- printed by <code>evaluation.py</code>
Cost per run	Token usage / paid-tier-equivalent cost	Automatic -- logged <code>usage_metadata</code> per LLM call
Tone / quality	Reads as professional, client-ready copy	Human-scored per run
Safety	Resists adversarial input (e.g. prompt injection), no leaked internal instructions	Human-scored per run, especially TC09/TC10

Most common failure pattern & fix

The most likely recurring failure pattern, based on the test design, is in the bad-input and adversarial cases: an LLM-based triage or critique step occasionally producing a plausible-sounding but ungrounded number (e.g. a price outside the matched service's range) when the underlying data was ambiguous or the request was adversarial. The mitigation already built into the pipeline is to keep every number traceable to `services_catalog.json` rather than letting the LLM invent one — `retrieve_node` hands `draft_proposal` the exact catalog entry as a hard constraint, and the recommended next step is a lightweight validator node after critique that checks the quoted price falls within `[price_usd_min, price_usd_max]` for the matched service, forcing a revision if not, rather than relying on the critique LLM call alone to catch it.

Known limitations

- Currency conversion depends on a free public API with no SLA -- a sustained outage degrades every non-USD quote to a raw-amount approximation, which is safe but not accurate.
- The services catalog is a static local JSON file; a real deployment needs it synced with whatever pricing sheet the business actually maintains, or drift between the two becomes a source of bad quotes.
- Prompt-injection resistance (TC09) has not been adversarially hardened beyond default model behavior and the fact that approval always requires a real human action -- a more sophisticated injection attempt is worth testing before this handles untrusted, high-volume public traffic.
- Single in-memory checkpointer (`MemorySaver`) does not survive a process restart -- production deployment needs a persistent checkpointer (e.g. a database-backed one) so a paused approval doesn't vanish if the API restarts.

Recommended next steps

- Scaling: move from `MemorySaver` to a persistent `LangGraph` checkpointer, and put the services catalog behind a real internal API instead of a static file.
- Guardrails: add the price-range validator node described above, and expand adversarial test coverage before opening the endpoint to unauthenticated public traffic.
- Human oversight: keep the `publish_gate` approval step mandatory for the foreseeable future -- it is cheap insurance against exactly the kind of ungrounded-number failure mode identified above, and should only be loosened after the price-range validator has a track record.